

# CYBR 437 Secure Coding

## Winter 2026

### Lab - 9

Release date: March 3, 2026

**Due date: March 9, 2026, at 11:59 pm**

## Introduction

This lab presents you with a number of common problems that might be solved using regular expressions, and you will write regular expressions for each of them. You are provided a Python file `cybr437regex.py`. Use Regext101 (<https://regex101.com/>) to test your patterns against a set of valid and invalid inputs. Once complete, add your final solution to the `cybr437regex.py` file and include 5 of your valid matches and 5 of your invalid matches in the test file `cybr437regextesting.py`, and then run the test to confirm that your pattern matches each test case appropriately.

**Note:** Make sure you document any cases you could not get to work, or any shortcomings for a given problem.

## Problem 1 (20 points):

Social Security Number (SSN) (can be with dashes, whitespaces, or no spaces at all)

NOTE: dashes and whitespace will only occur between places 3 and 4 and/or between places 5 and 6. The rules are:

- a) No valid SSN has all zeros in any of the 3 segments
- b) No valid SSN has 9 identical digits or has the 9 digits running consecutively from 1-9
- c) SSA will issue SSNs with the digits “1”, “8”, and “9” only in position 1
- d) SSA will not issue SSNs beginning with the digit “9” unless the position 4 digit is a 9
- e) SSA will not issue SSNs beginning with the digits “666” in positions 1-3
- f) The SSN must be exactly 9 digits

### - Valid Examples

- 132456789
- 132-45-6789
- 143 45 6789
- 132-45 6789
- 135 45-6789
- 13345-6789

### - Invalid Examples

- 123-45-6789
- 12-345-6789

- 123--45-6789
- 123.45.6789
- 111-11-1111
- 000-26-6781

## Problem 2 (20 points):

E-mail address – must contain @ and more than 2 characters after the at symbol. A dot can be present in a string before the @ symbol, but it is not allowed as the character right before the @ symbol. A dot cannot be used more than once consecutively.

- Valid Example
  - john.smith@ewu.edu
- Invalid Examples
  - john.smith.@ewu.edu
  - mr.john.smith@ewu.edu

## Problem 3 (20 points):

Date in MM-DD-YYYY format

You need to account for the following:

- Separators can be either the dash (-) or the forward slash (/) but not both
- An optional leading zero
- Must be a valid date, for example, 3/32/2023 is not valid

## Problem 4 (20 points):

A password that contains at least 10 characters and includes at least one upper case character, one lower case character, one digit, one punctuation mark, and does not have more than 3 consecutive lowercase characters or two digits that repeat. The password must start with a lowercase character or a punctuation mark. For example, Pa\$sword1212 is invalid.

## Problem 5 (20 points):

All words containing an even number of letters, ending with “ion” (case-insensitive).

- Valid Examples:
  - Lion
  - conversIon
  - 0rion

### **Submission Instructions**

Your version of the two Python files

Your write-up submission should include:

- The regular expressions for each problem
- Detailed explanation of how each part of a regular expression exactly handles the constraints in the problem
- 5 passing tests and 5 failing tests for each task, along with screenshots of the test and output